

Sklearn Preprocessing Functions

Standardization, Normalization,
Categorical Encoding, and Outlier
Detection

Standardization

- Standardization scales features to have zero mean and unit variance.
- Used when data has varied scales or units.
- Example: `StandardScaler()` from `sklearn.preprocessing`
- Example:

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Normalization

- Normalization scales features to a range, usually $[0, 1]$.
- Helpful for algorithms that require bounded input values.
- Example: `MinMaxScaler()` from `sklearn.preprocessing`
- Example:

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
X_normalized = scaler.fit_transform(X)
```

Categorical Encoding

- Transforms categorical data to numerical format.
 - One-Hot Encoding and Label Encoding are common methods.
 - Example: `OneHotEncoder()` from `sklearn.preprocessing`
- Example:

```
from sklearn.preprocessing import OneHotEncoder
encoder = OneHotEncoder()
X_encoded = encoder.fit_transform(X)
```

Outlier Detection

- Detects and removes or flags outliers in the data.
 - Helps prevent model skew due to anomalous data points.
 - Example: IsolationForest from sklearn.ensemble
- Example:

```
from sklearn.ensemble import IsolationForest
clf = IsolationForest()
outliers = clf.fit_predict(X)
```

Missing Values and Imputation

- Imputation fills missing values to prevent data loss.
- Mean, median, and mode are common strategies.
- Example: SimpleImputer from sklearn.impute

- Example:

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(X)
```

Label Encoding

- Label Encoding assigns a numerical value to each category.
- Useful for ordinal categorical data.
- Example: LabelEncoder from `sklearn.preprocessing`
- Example:

```
from sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()
y_encoded = encoder.fit_transform(y)
```

Datasets in Sklearn

- Scikit-learn includes a variety of built-in datasets.
- Useful for practice and experimentation.
- Examples: iris, digits, boston, wine, breast_cancer

Loading Datasets

- Datasets can be loaded with sklearn's data fetching functions.
- Each dataset has a consistent structure (features, targets, descriptions).
- Example: `load_iris()`

- Example:

```
from sklearn.datasets import load_iris
iris = load_iris()
X, y = iris.data, iris.target
```

Custom Data Loading

- For external datasets, use pandas or numpy for flexible loading.
 - Data can be loaded from CSV, Excel, databases, etc.
 - Example: Using pandas to load a CSV file
- Example:

```
import pandas as pd
data = pd.read_csv('data.csv')
X = data.iloc[:, :-1]
y = data.iloc[:, -1]
```

Exploring Data After Loading

- Always explore the data with head, describe, and info.
- Check for missing values and anomalies.
- Example: `data.head()`, `data.describe()`, `data.info()`

- Example:
`data.head()`
`data.describe()`
`data.info()`